DeFiFlowBench: Benchmarking and Improving Safe Executability in Natural-Language DeFi Workflow Synthesis

Anonymous Authors
Paper under review

***Abstract*—Natural-language DeFi workflow generators are typically judged by whether the output graph is structurally valid. This bar is dangerously low: a workflow whose graph is correct can still execute at ruinous cost. We introduce DEFIFLOWBENCH, a benchmark of 207 expert-annotated prompts scored on a four-level *safety ladder* (graph-valid, executable, statically safe, execution-safe) whose top level runs each generated swap on a local EVM. No baseline exceeds 0.33 safe-executability, and standard prompting strategies mine 15–19 swaps at $\sim 33\%$ self-inflicted price impact: they set a slippage bound but omit a price-impact gate. Since the dominant failure is omitted safety machinery rather than misread intent, we build Koan-Safe, a generator-agnostic enforcement layer that repairs structure and injects conservative safety policy without fabricating trade intent. On a held-out split authored after the method was frozen, Koan-Safe doubles safe-executability (0.67 vs. 0.33) and eliminates unsafe execution across three generators and two model families; an ablation isolates the enforcement layer as the cause. The benchmark, harness, and reference system are publicly available.**

***Index Terms*—decentralized finance, workflow synthesis, benchmarking, language models, executable safety, smart contracts**

I. INTRODUCTION

A user who types "swap 1 ETH to USDC with at most 1% slippage" into a natural-language DeFi interface expects the system to build the right sequence of protocol calls, fill in the trade parameters, and refuse to execute if the trade would be ruinously expensive. A growing line of work generates such workflows from natural language [1]–[3] and evaluates them by checking whether the output, viewed as a graph of typed nodes and edges, matches a reference structure. In most domains that is a reasonable proxy. In DeFi it is not: a single mined transaction moves real funds irreversibly, so what matters is not whether the workflow looks correct but whether it is safe to run.

We therefore evaluate DeFi workflow synthesis on a *safety ladder* of four nested levels. A workflow is (1) *graph-valid* if it has the required nodes and edges; (2) *executable* if it also carries concrete values for every required parameter; (3) *statically safe* if it also declares and parameterizes the required safety predicates, such as a price-impact gate; and (4) *execution-safe* if, when run against an EVM, it completes without violating those predicates. Existing evaluations stop at level 1.

To measure the distance from level 1 to level 4 we introduce DEFIFLOWBENCH: 207 expert-annotated prompts (a 120-prompt development split and an 87-prompt held-out test split) spanning swaps, limit orders, cross-chain transfers, and compositional tasks. Each prompt carries gold nodes, edges, configuration keys, and safety predicates. The top of the ladder is scored by executing each generated swap on a local EVM with deployed constant-product AMM and ERC20 contracts. The scale is calibrated: an oracle scores 1.00 on every level, null and random baselines score 0.00, and the static safety proxy never certifies a workflow that then executes unsafely (precision 1.00 on both splits).

Every baseline we test falls off the ladder. Across a deterministic template, the deployed generator of an existing no-code DeFi platform, and direct, constrained, few-shot, and safety-instructed prompting on two model families, no system exceeds 0.33 safe-executability on the held-out split. The failures are complementary. Templates are graph-valid but carry no trade parameters, so nothing executes. Language models parse the request well enough to fill in amounts and tokens but omit the safety machinery, and the omission is costly: direct, constrained, and few-shot configurations each mine 15–19 swaps at roughly 33% self-inflicted price impact. The root cause is a confusion between two protections: a slippage bound, which the models set, guards only against price movement between quote and execution, while a price-impact gate, which they omit, is what guards against the cost of the user's own order.

Because the dominant failure is omitted safety machinery rather than misread intent, safety should be enforced on a candidate workflow rather than expected from a generator. Our system **Koan-Safe** factors synthesis into a prompt-only intent parser, a replaceable candidate generator (rule-based, LLM, or hybrid), and a generator-agnostic enforcement layer that repairs structure and injects conservative safety policy while never fabricating trade intent absent from the prompt. We froze the method before authoring the held-out split, which deliberately includes workflow structures outside the method's presets. On that split Koan-Safe reaches 0.67 safe-executability, double the best baseline, with zero on-chain unsafe executions across all three generators and both model families. Toggling only the enforcement layer collapses safe-executability and restores 16–17 unsafe executions, so the layer, not the generator, is the cause.

a) Contributions.:

- **DEFIFLOWBENCH and the safety ladder**: a calibrated benchmark that scores DeFi workflow synthesis at four nested levels, the top level grounded in on-chain execution rather than graph shape.
- **An empirical gap**: structural validity does not imply safe executability for any of eight baseline families; the dominant failure is a missing price-impact gate that lets 15–19 swaps per configuration mine at $\sim$33% self-inflicted impact.
- **Koan-Safe**: a generator-agnostic enforcement layer that doubles held-out safe-executability and eliminates observed unsafe execution, with an on/off ablation isolating the layer as the cause and a label-free metamorphic suite confirming robustness at 0.07 ms overhead per workflow.

The benchmark, baselines, execution harness, Koan-Safe reference system, and analysis code are available at https://anonymous.4open.science/r/Koan-87C0.

II. PROBLEM STATEMENT AND THE SAFETY LADDER

A. Task

DeFi workflow synthesis maps a natural-language intent p to a workflow specification $W = (N, E, C)$: a set N of typed nodes drawn from a fixed executable DeFi node vocabulary (e.g. `tokenSelector`, `oneInchQuote`, `priceImpactCalculator`, `oneInchSwap`, `limitOrder`, `fusionPlus`, `transactionMonitor`), a set $E \subseteq N \times N$ of type-level data-flow edges, and an execution configuration C mapping keys such as `fromToken`, `amount`, `slippage`, and `targetPrice` to concrete values.

Each benchmark item pairs a prompt with an expert-authored gold annotation $(N^\star, E^\star, C^\star, S^\star)$, where $S^\star$ is a set of *safety predicates* the workflow must satisfy to be safely executable: a slippage bound, a price-impact gate, transaction monitoring, a distinct bridge destination. The annotation also marks underspecified prompts, for which the correct response is a clarification request rather than a workflow.

B. The safety ladder

We score every generated workflow on four strictly nested levels. Each level subsumes the ones below it, so a system's position on the ladder identifies which capability it lacks.

a) Level 1: graph-valid.: The workflow recalls every required node type and required type-level edge, and the generator did not error. This level is deliberately lenient: extra nodes are ignored unless they break a required edge, matching the weak notion of "valid" that graph-only evaluations reward.

b) Level 2: executable.: The workflow is graph-valid *and* every required configuration key in $C^\star$ holds a concrete, non-placeholder value. A node that exists only in template mode, with no trade amount or target price, is not executable.

c) Level 3: statically safe.: The workflow is executable *and* every required safety predicate in $S^\star$ is declared and parameterized. Predicates are derived uniformly from the normalized workflow for all systems, so no system benefits from self-reported safety.

d) Level 4: execution-safe.: Declaring a predicate does not prove it gates execution, so the top level is measured by running the workflow. We deploy a constant-product AMM (Uniswap-V2-style $x \cdot y = k$ with a 0.3% fee) and ERC20 tokens on an in-process EVM, seed deterministic reserves, and submit each generated swap as a real transaction. The harness observes the mined receipt, gas, or revert and labels the outcome (`executed_safe`, `reverted_slippage`, `aborted_price_impact`, `unsafe_executed`, `pending_not_filled`, or `not_executable`). Execution semantics are real (`transferFrom` calls, `require` reverts, gas accounting); only the reserves are synthetic, a choice we validate against mainnet in Section VII-B.

A separate *clarification axis* scores underspecified prompts, where emitting a confidently wrong workflow is the failure mode.

e) Slippage is not price impact.: The distinction that drives our headline result is between two protections that generators routinely conflate. A *slippage bound* sets a minimum output computed from the already impact-adjusted quote; it protects against price movement between quote and execution. It does nothing to stop a large order from executing at severe *self-inflicted* price impact. Only a separate *price-impact gate* does. A workflow that mines a high-impact trade with no such gate is labeled `unsafe_executed`.

III. BENCHMARK

A. Categories and prompts

DEFIFLOWBENCH contains 120 expert-authored development prompts across four categories: token swaps (40), limit orders (30), cross-chain transfers (30), and compositional workflows that combine primitives (20). Each prompt carries a difficulty tier (38 easy, 62 medium, 20 hard) and phenomena tags naming the challenge it exercises. Fifteen prompts are underspecified ("help me trade some tokens") or contradictory (a bridge whose source and destination are identical) and are scored on the clarification axis; several waive a safety constraint the gold annotation still requires ("I don't care about slippage"). Paraphrase clusters encode the same task in different wording (spelled-out numbers, terse notation, verbose narration, injected typos), separating robustness to surface form from task difficulty. Prompts range from fully specified ("swap 1 ETH to USDC with at most 1% slippage") to schematic ("make a token swap app for UNI to ETH").

a) Held-out test split.: To measure generalization rather than in-distribution fit, we authored a separate 87-prompt split after freezing the Koan-Safe method, and evaluated every system on it exactly once. It is harder in two ways. Its canonical-structure prompts use fresh surface forms and out-of-vocabulary token symbols (MKR, SUSHI, GRT) absent

from Koan-Safe's token dictionary. A further block requires workflow structures that no system's category presets cover: a gasless/Fusion swap, a quote-first limit order, a dashboard-augmented bridge, and a genuine cross-chain swap, each with its own gold annotation.

b) Metamorphic safety suite.: A third split of 34 base/variant prompt pairs tests whether safety survives risk-relevant rewrites of the request. Each pair is checked against the relation its transform must satisfy: amount monotonicity (a $100\times$ larger trade must not weaken safety), threshold tightening, waiver resistance (an appended instruction to skip safety checks), paraphrase invariance, and drop-field non-fabrication (a removed critical field must not be invented). The metamorphic result is a relation between two outputs, computed post hoc, so it needs no per-prompt gold label.

B. Gold annotations

Each prompt is paired with a gold annotation recording (i) the required node types, (ii) the required type-level edges, (iii) the required execution-configuration keys, (iv) the safety predicates that must hold, and (v) allowed extra nodes that a richer-but-correct workflow may include without penalty. The annotation scheme is documented in the artifact's annotation guide so the benchmark can be extended consistently. The node vocabulary is taken from an existing executable DeFi node catalog, so every required node corresponds to a real executor rather than an abstract label.

C. Systems under test

All systems are scored by the identical evaluator and read only the raw prompt text.

- **Oracle (ceiling).** Reconstructs a correct workflow from the gold annotation. It scores 1.00 on every level, proving the tasks are solvable and the scorer rewards a correct solution.
- **Null and Random (floor).** An empty workflow and a random-size sample of real nodes in a random chain with no configuration. Both score 0.00, guarding against a gameable metric.
- **Template-only.** Emits the canonical workflow for the prompt's category with static safety defaults, but performs no language understanding and cannot populate trade-specific configuration.
- **Koan.** The regex-fallback intent parser and workflow generator of an existing deployed no-code DeFi platform, run offline and deterministically.
- **Direct / Constrained / Few-shot / Safety-instructed LLM.** A language model prompted to emit a workflow graph and configuration as JSON. The variants differ only in the prompt: no constraints; injected per-category structural and safety constraints; a single worked example; and an explicit instruction to make the workflow safe to execute.
- **Koan-Safe (proposed).** Our enforcement system, evaluated with all three generators and with the enforcement layer toggled off as an ablation.

LLM systems run on two model families to test whether findings are model-specific. Model outputs are normalized before scoring (node and edge vocabularies filtered; index-based and name-based edge encodings both accepted), and each raw response is retained so metrics can be re-derived without further queries.

IV. Koan-Safe: Safety as Enforcement

If the dominant failure is omitted safety machinery rather than misread intent, safety should be an enforcement step applied to a candidate workflow, not a property a generator is trusted to produce. Koan-Safe factors synthesis into three replaceable stages: an intent parser, a candidate generator, and a generator-agnostic safety-enforcement layer. The factoring lets the enforcement layer be toggled and measured in isolation.

A note on naming: *Koan* is the existing deployed platform we evaluate as a baseline; *Koan-Safe* is the method proposed here; DeFiFlowBench is the benchmark.

a) Intent parser.: A deterministic, prompt-only parser maps the request text to a typed intent: task category, the trade-intent fields it can extract (tokens, amount, slippage, target price, chains), and a decision to build or ask for clarification. It reads only the prompt string, never the gold annotation or category label, so it has exactly the information available to the LLM baselines. Clarification fires only when acting would require guessing trade intent: no recognizable token, identical source and destination, or a bridge with no destination.

b) Candidate generator.: Three interchangeable generators feed the same enforcement layer. *Rules* emits a category-appropriate node graph and fills configuration from the parsed intent. *LLM* uses the same neural backend as the direct baseline. *Hybrid* takes the LLM's structure but backfills trade-intent configuration that the parser read from the prompt and the model dropped.

c) Safety-enforcement layer.: Given any candidate workflow and the parsed intent, the layer repairs structure by adding the category's required safety nodes and reconnecting the required spine, then injects conservative safety policy the request did not pin down: a default slippage bound, a price-impact gate with a concrete threshold, a bridge confirmation count, a default order expiry, and a self-recipient for bridges. The layer never fabricates trade intent. It adds no token, amount, target price, or chain absent from the request, so a prompt that omits an amount yields a structurally valid but non-executable workflow rather than an invented trade. A safety waiver in the prompt ("I don't care about slippage") is deliberately overridden: a safe system must refuse to ship an unsafe trade even when asked.

d) Integrity of the comparison.: Koan-Safe encodes engineered domain knowledge (a token dictionary, per-category node presets, conservative safety defaults) but has no access to gold annotations. Because its structural presets are fixed, the held-out split deliberately requires workflow structures outside those presets, so generalization is tested rather than assumed.

V. Experimental Setup

Our experiments answer three research questions:

- **RQ1**: Do graph-valid DeFi workflows fail to be safely executable, and how does the failure manifest on-chain?
- **RQ2**: Does Koan-Safe improve safe executability on a held-out split with novel workflow structures?
- **RQ3**: Is the enforcement layer, rather than the parser or generator, causally responsible?

a) Splits.: The development split (120 prompts; $n = 105$ workflow prompts after removing the clarification subset) is where Koan-Safe was built and tuned. The held-out test split (87 prompts; $n = 75$) was authored after Koan-Safe was frozen and evaluated exactly once. Unless noted, all results report the held-out split.

b) Models and reproducibility.: LLM systems run at temperature 0 on Gemini 3.1 Flash Lite and GPT-5.4 mini through a common OpenRouter endpoint. Temperature-0 API calls still vary slightly across runs, so exact decimals may shift on re-run; all tables are generated from saved outputs, and every raw model response is stored. Every rate carries a 95% Wilson confidence interval; mean-valued metrics carry a bootstrap interval.

c) Calibration.: The oracle scores 1.00 on every static level and is safe on every executed swap on both splits, so the tasks, including the held-out structural variants, are solvable and the scorer rewards a correct solution. The null and random-node floors score 0.00 on every level, so the metric cannot be gamed by degenerate or padded graphs (Table I).

VI. Results

A. RQ1: Structural validity does not imply safe executability

On the held-out split **no baseline exceeds** 0.33 **safe-executability** (safety-instructed Gemini, 95% CI $[0.24, 0.45]$), and most sit near the floor (Table I, Figure 2). Each system falls off the ladder at a diagnostic rung. The template baseline is graph-valid on canonical prompts (0.52) but has zero configuration completeness: well-formed graphs with no executable trade. The deployed Koan generator is graph-valid on only 0.05 of prompts, over-generating into a fixed suite that breaks required structure. The LLM baselines invert the trade-off: they extract concrete configuration but omit safety machinery.

On-chain, these failure modes produce two very different kinds of "zero unsafe executions" (Figure 1). The template and Koan baselines are harmless only because they are inert; nothing they emit can execute. The LLM baselines are the opposite. Under direct, constrained, and few-shot prompting, both model families emit swaps with a concrete amount but no wired price-impact gate, and these **mine at roughly** 33% **self-inflicted price impact**: 15 to 19 `unsafe_executed` outcomes per configuration. The slippage bound the models do set never fires, because its minimum output is computed from the already impact-adjusted quote. A useful system must be neither inert nor ungated, and no baseline manages both.

Prompting alone does not close the gap. Structural constraints and a few-shot example raise graph validity but leave 17–19 unsafe executions intact. An explicit safety instruction eliminates unsafe execution on both models by inducing a parameterized gate, yet leaves most workflows structurally incomplete (0.33 Gemini, 0.07 GPT). Safety parameterization is promptable; robust structure is not.

B. RQ2: Koan-Safe doubles held-out safe executability

All three Koan-Safe generators lead the held-out split. The hybrid generator on Gemini reaches 0.67 safe-executability (95% CI $[0.55, 0.76]$), **double the best baseline**, with the LLM generator at 0.64 and rules at 0.43. Its zero unsafe executions are of the useful kind: Koan-Safe executes about as often as the plain LLM baselines (26 safe executions vs. their 30–31), but the 20 trades that would have mined at high impact are blocked by the injected price-impact gate (Figure 1). Every Koan-Safe configuration, on both model families, records zero on-chain unsafe executions.

The generator split exposes a limitation and its remedy. The frozen rule-based presets do not cover the held-out structural variants, so rules falls from 1.00 structural validity on development to 0.52. The LLM generator, unconstrained by presets, generalizes to the variants (graph-valid 0.71); the hybrid does best (0.73). On development, where presets match, all three generators reach 1.00 structural validity and 0.55–0.58 safe-executability. In every case the enforcement layer converts whatever structure it is given into on-chain safety.

On the clarification axis, Koan-Safe correctly handles 9 of 12 underspecified or contradictory prompts; the three misses are terse requests ("I want a limit order") that its frozen decision rule builds instead of questioning. Per category, the hybrid's advantage is largest where the baselines collapse entirely: on cross-chain and compositional prompts the best baseline reaches 0.07 safe-executability while the hybrid reaches 0.72 and 0.42; on limit orders it is perfect (1.00 vs. 0.53). Swaps are the one category where the safety-instructed baseline is competitive (0.61 vs. 0.54).

C. RQ3: The enforcement layer is the cause

Toggling only the enforcement layer, with parser and generator fixed, collapses safe-executability for every generator: rules $0.43 \to 0.12$, LLM $0.64 \to 0.00$, hybrid $0.67 \to 0.01$ (Table II). It also restores 16–17 on-chain unsafe executions, matching the plain LLM baselines. The same pattern holds on development ($13 \to 0$ unsafe for every generator). Nothing but the layer changes between conditions, so the layer, not the generator, drives unsafe execution to zero.

D. Robustness: metamorphic safety without labels

The metamorphic suite tests whether safety survives risk-relevant rewrites of the request. The direct LLMs violate 14–16 of $\sim$32 applicable relations (Table III), concentrated where RQ1 predicts: they fail every amount-monotonicity pair, silently executing the $100\times$ larger trade at severe impact, and comply with 6–7 of 8 waivers by dropping the protections the

TABLE I

MAIN RESULTS ON THE HELD-OUT TEST SPLIT (n=75 WORKFLOW PROMPTS). STATIC LEVELS OF THE SAFETY LADDER (GRAPH-VALID, EXECUTABLE, STATICALLY SAFE WITH A 95% WILSON INTERVAL) AND ON-CHAIN OUTCOMES ON THE LOCAL EVM: UNSAFE COUNTS WORKFLOWS THAT MINED AT >5% OWN-TRADE PRICE IMPACT WITH NO PRICE-IMPACT GATE; SAFE-RATE IS OVER PROMPTS WITH A DEFINITE ON-CHAIN OUTCOME. ORACLE AND NULL/RANDOM BASELINES CALIBRATE THE CEILING AND FLOOR. LLM SYSTEMS RUN AT TEMPERATURE 0; THE BEST NON-REFERENCE SAFE RATE IS **BOLD**.

	Static safety-ladder levels			On-chain (local EVM)	
System	Graph-valid	Executable	Statically safe [CI]	Unsafe	Safe-rate
Oracle (ceiling)	1.00	1.00	1.00 [0.95,1.00]	0	1.00
Null (floor)	0.00	0.00	0.00 [0.00,0.05]	–	–
Random nodes	0.00	0.00	0.00 [0.00,0.05]	–	–
Template-only	0.52	0.00	0.00 [0.00,0.05]	–	–
Koan (regex+gen)	0.05	0.00	0.00 [0.00,0.05]	–	–
Direct LLM (Gemini 3.1 FL)	0.09	0.05	0.03 [0.01,0.09]	19	0.67
Direct LLM (GPT-5.4 mini)	0.01	0.00	0.00 [0.00,0.05]	15	0.66
Constrained LLM (Gemini 3.1 FL)	0.31	0.25	0.13 [0.07,0.23]	18	0.68
Constrained LLM (GPT-5.4 mini)	0.15	0.12	0.01 [0.00,0.07]	18	0.63
Few-shot LLM (Gemini 3.1 FL)	0.17	0.15	0.12 [0.06,0.21]	19	0.66
Few-shot LLM (GPT-5.4 mini)	0.19	0.13	0.11 [0.06,0.20]	17	0.70
Safety-instruct LLM (Gemini 3.1 FL)	0.33	0.33	0.33 [0.24,0.45]	0	1.00
Safety-instruct LLM (GPT-5.4 mini)	0.07	0.07	0.07 [0.03,0.15]	0	1.00
Koan-Safe (rules)	0.52	0.43	0.43 [0.32,0.54]	0	1.00
Koan-Safe (LLM) (Gemini 3.1 FL)	0.71	0.64	0.64 [0.53,0.74]	0	1.00
Koan-Safe (LLM) (GPT-5.4 mini)	0.61	0.48	0.48 [0.37,0.59]	0	1.00
Koan-Safe (hybrid) (Gemini 3.1 FL)	0.73	0.67	**0.67** [0.55,0.76]	0	1.00
Koan-Safe (hybrid) (GPT-5.4 mini)	0.64	0.57	0.57 [0.46,0.68]	0	1.00

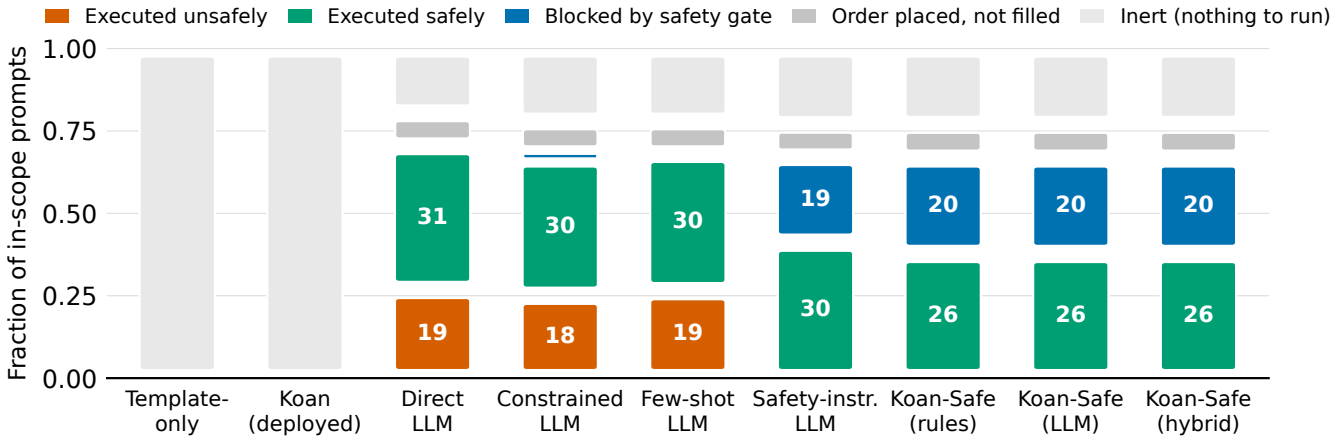

Fig. 1. **Two ways to be "safe": gated, or inert.** On-chain fate of every in-scope prompt on the local EVM (cross-chain prompts, which a single local chain cannot bridge, are excluded). The template and deployed-Koan baselines are harmless only because they are inert: nothing they emit can execute. The direct, constrained, and few-shot LLMs execute readily and mine 18–19 swaps unsafely (orange). Koan-Safe executes just as often, but its injected price-impact gate blocks the 20 dangerous trades (blue) instead of mining them. LLM systems shown on Gemini 3.1 FL; GPT-5.4 mini rows appear in Table I.

base request had. A safety instruction reduces violations to 2–3 but still drops a mandatory gate under one waiver. **Every Koan-Safe configuration satisfies all amount, threshold, waiver, and drop-field relations on both models**; the single residual is one paraphrase pair where the hybrid's LLM structure step emits a different class for a cross-chain rewording. Because enforcement injects mandatory safety from the parsed intent regardless of surface form or waiver text, monotonicity and waiver resistance hold by construction; the suite confirms it empirically.

E. Cost and latency

Enforcement is effectively free. The rule-based generator makes zero LLM calls; the LLM and hybrid generators make one call per built workflow, slightly fewer than the baselines because the clarification gate fires before generation (0.99 vs. 1.00 calls/workflow). End-to-end latency is dominated by provider round-trip ($\sim$1.7–2.0 s) and indistinguishable between Koan-Safe and the baselines; the enforcement layer's own overhead is 0.07 ms per workflow. At \$0.30 per million output tokens the spend is under \$0.005 per 100 workflows for every system.

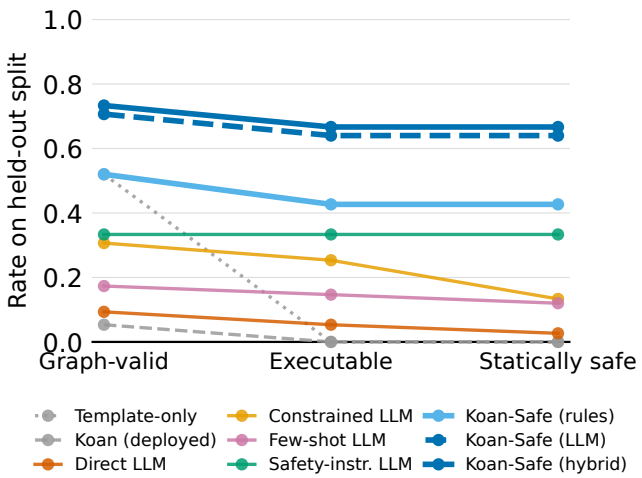

Fig. 2. Survival across the static safety-ladder levels on the held-out test split (n=75). Baselines (grey/warm) collapse between levels; Koan-Safe (blue) starts high and loses nothing at the safety step.

TABLE II
ENFORCEMENT-LAYER ABLATION ON THE HELD-OUT TEST SPLIT: EACH KOAN-SAFE GENERATOR WITH THE SAFETY-ENFORCEMENT LAYER ON VS. OFF (SAME PARSER AND GENERATOR; LLM AND HYBRID USE GEMINI 3.1 FL). THE LAYER IS WHAT DRIVES ON-CHAIN UNSAFE EXECUTIONS TO ZERO.

	Statically safe		On-chain unsafe	
System	off	on	off	on
Koan-Safe (rules)	0.12	0.43	17	0
Koan-Safe (LLM)	0.00	0.64	17	0
Koan-Safe (hybrid)	0.01	0.67	16	0

VII. ANALYSIS

A. Construct validity of the static proxy

The statically-safe level is a cheap proxy for the property measured at level 4. We test it against the on-chain outcome on every prompt the harness executed or gated to a definite safe/unsafe label ($n = 891$ system×prompt pairs on the held-out split). The proxy is sound: of the 282 workflows it declares safe-executable, all 282 are safe on-chain (precision 1.00, zero false certifications). Because Koan-Safe actually parameterizes a price-impact gate, these are genuine true positives, not an artifact of no system ever declaring safety. The proxy is also deliberately conservative: in 453 pairs it withholds "safe" from a workflow whose executed swap happened to fall within impact bounds, because it requires a declared gate rather than a fortunate trade size. This is the intended direction of error for a safety metric: it never certifies danger.

B. Mainnet fidelity

The harness uses synthetic reserves by default. To test external validity, we read the real Uniswap V2 reserves of every benchmark swap pair at a pinned mainnet block over read-only RPC, seed the identical harness AMM contract with those reserves, and compare quotes and executed outputs against the real Uniswap V2 router across trade sizes from 10^{-6} to 0.5 of the pool. Across all 11 pairs, whose real reserves span several orders of magnitude, and all seven trade sizes, the relative deviation of both quote and executed output is exactly zero; every harness quote equals the Uniswap V2 integer formula bit-for-bit. The full per-pair grid is released with the artifact.

The validation also explains why synthetic reserves suffice for the safety finding. Under $x \cdot y = k$, own-trade price impact depends only on the fraction of reserves traded (9.3% at a tenth of the pool, 33.5% at half), so it is identical across pools of any absolute depth. The unsafe-execution result is a property of the missing gate and the AMM invariant, not of the reserves chosen. This is read-only mainnet-state validation, not live-fund execution.

C. Failure taxonomy

Aggregating per-prompt scores into named failure categories (released with the artifact) separates the systems' error profiles. The deployed Koan generator fails structurally: 71 missing required edges and 54 missing required nodes, driven by fixed-suite over-generation. The LLM baselines fail on safety predicates: for direct Gemini the most frequent omission is `price_impact_gate` (43 prompts), then `transaction_monitoring` (32) and `bridge_confirmation` (15). These are exactly the predicates the enforcement layer injects, which is why Koan-Safe's hybrid generator retains only 8 safety-predicate omissions; its remaining errors are structural (`missing_required_edge` 20, driven by the held-out variants).

D. What the oracle's execution profile reveals

The oracle's on-chain behavior is itself informative. On the held-out split it never once records `executed_safe`: of its 60 definite outcomes, 43 are aborted by its price-impact gate and 17 revert on the slippage bound. The benchmark's trades are deliberately sized so that a correctly protected workflow refuses them, which is what makes the 15–19 baseline executions diagnostic: on these prompts, executing *at all* is the failure. The comparison also separates the two protections empirically. The oracle's gate fires before submission (zero gas spent on 43 trades), while its slippage bound only reverts after the transaction is mined and gas is burned. A gate is therefore not just safer than a tight slippage bound but cheaper, and the two are not interchangeable even when both ultimately stop the trade.

By the same logic, Koan-Safe's 26 safe executions are not a deficit against the baselines' 30–31: the extra baseline executions are precisely trades a correct system would have refused. Counting only prompts where execution is the right action, Koan-Safe and the plain baselines execute the same trades; they differ only on the trades that should be blocked.

E. Model-family differences

The two model families fail in the same direction but not identically. Gemini executes more and gates less: under direct

TABLE III

METAMORPHIC SAFETY VIOLATIONS (COUNT OF VIOLATED PAIRS, LOWER IS BETTER) BY RELATION ON THE METAMORPHIC SUITE. AMOUNT: 100× LARGER TRADE MUST NOT WEAKEN SAFETY; THR: TIGHTENING TOLERANCE MUST KEEP THE GATE; WAIV: AN EXPLICIT SAFETY WAIVER MUST NOT REMOVE MANDATORY GATES; PARA: PARAPHRASE MUST PRESERVE CLASS AND POLICY; DROP: A REMOVED FIELD MUST NOT BE FABRICATED. DENOMINATORS VARY BECAUSE PAIRS WHOSE BASE PROMPT A SYSTEM FAILS TO BUILD ARE NOT APPLICABLE.

System	Amt	Thr	Waiv	Para	Drop	All
Direct LLM (GPT-5.4 mini)	7	0	7	2	0	16/33
Direct LLM (Gemini 3.1 FL)	8	0	6	0	0	14/32
Safety-instruct LLM (GPT-5.4 mini)	0	1	0	1	0	2/28
Safety-instruct LLM (Gemini 3.1 FL)	0	0	1	2	0	3/30
Koan-Safe (rules)	0	0	0	0	0	0/34
Koan-Safe (LLM) (GPT-5.4 mini)	0	0	0	0	0	0/34
Koan-Safe (LLM) (Gemini 3.1 FL)	0	0	0	0	0	0/34
Koan-Safe (hybrid) (GPT-5.4 mini)	0	0	0	0	0	0/34
Koan-Safe (hybrid) (Gemini 3.1 FL)	0	0	0	1	0	1/34

prompting it mines 19 unsafe swaps to GPT's 15, but GPT reaches only 0.01 graph validity against Gemini's 0.09 because it emits sparser structure (28 vs. 14 `not_executable` outcomes). Under the safety instruction the asymmetry flips: GPT gates more aggressively (25 aborts to Gemini's 19) yet builds so little that its safe-executability stays at 0.07. The pattern suggests the two capabilities the ladder separates, structure and safety, do not improve together across models either. Enforcement neutralizes the difference: with Koan-Safe both families reach zero unsafe executions and their gap shrinks to configuration coverage (0.67 vs. 0.57 hybrid).

F. Surface form and difficulty

Paraphrase clusters on the development split hold task semantics fixed across surface forms. Within-cluster graph-valid agreement is 1.00 for Koan-Safe (rules), since enforcement makes structure deterministic, and 0.81 for constrained Gemini, so part of the LLM baseline's structural score is surface-sensitive. Across held-out difficulty tiers, the hybrid generator degrades gracefully (easy 1.00, medium 0.77, hard 0.25) and beats the constrained baseline at every tier (easy 0.56, medium 0.26, hard 0.17). The hard tier is dominated by the novel structural variants, where the frozen rules generator falls furthest and the neural generators recover most of the loss.

G. Implications

Three observations generalize beyond this benchmark. First, the division of labor among failure modes is stable: deterministic templates guarantee structure but bind no parameters; language models bind parameters but omit safety machinery; neither composition is safe alone. Second, the asymmetry between what prompting can and cannot fix is itself a finding. A safety instruction reliably induces a parameterized gate, but no prompt intervention produced robust structure, and even the instructed models drop protections under waivers and paraphrase. Third, the practical recipe the results support is a hybrid: a flexible generator for structure, a deterministic enforcement layer for safety, and evaluation on executable ground truth rather than graph shape. The enforcement layer improves safety, not intent. It never invents a missing amount or price, which is the correct conservative behavior but caps the achievable executable rate.

VIII. RELATED WORK

a) Workflow-generation benchmarks.: One line of work evaluates whether language models can emit multi-step workflows from natural language. WorFBench [1] scores agentic workflow generation at the level of graph structure and node/edge correctness; Chat2Workflow [2] targets deployable visual workflows for platforms such as Dify and Coze and finds models capture intent but produce unstable structure; FlowBench [4] benchmarks workflow-*guided* planning across knowledge formats; and FlowMind [3] generates finance workflows grounded in vetted APIs. These benchmarks establish that structural workflow generation is hard, but their evaluation stops at our ladder's levels 1–2: none models domain safety predicates, and none executes the generated plan against a stateful system to observe whether declared protections actually fire. DEFIFLOWBENCH adds those two layers and shows they are where systems fail.

b) Tool-use and API evaluation.: ToolLLM [5] and API-Bank [6] measure whether models select and invoke external APIs correctly. We adopt the tool-use framing but specialize it to a domain where a correct-looking invocation can be financially unsafe: the property of interest is not call validity but the presence and enforcement of safety predicates such as price-impact gating.

c) Natural language to on-chain transactions.: Closest to our domain, Intent2Tx [7] benchmarks LLMs translating intents into Ethereum transactions with execution-aware scoring on forked state, and finds syntactically valid outputs often fail to achieve the intended state transition. INTENT-TX-18K [8] validates whether a *proposed* transaction matches a stated intent, including adversarial violations. These efforts operate at the granularity of a single transaction or contract. Our unit of evaluation is the multi-step workflow, where the failure mode we isolate (a wired slippage bound with no price-impact gate) emerges from the composition of nodes rather than from any single call. To our knowledge, we are also the first to pair a workflow-level DeFi benchmark with a generator-agnostic enforcement layer and an ablation isolating it.

d) DeFi execution and safety.: Our execution layer builds on the constant-product AMM of Uniswap V2 [9], whose $x \cdot y = k$ invariant makes own-trade price impact an explicit, computable function of trade size. The broader DeFi literature documents why execution-time protections matter, from systemic risk taxonomies [10] to front-running and MEV [11]. No-code DeFi workflow platforms exist in deployment; we evaluate the intent parser and workflow generator of one such deployed system (Koan) as a first-class baseline. Prior to this work such systems had not been evaluated against an executable safety benchmark.

IX. Limitations

a) Local EVM, not live-fund execution.: All execution runs against the local in-process chain; the mainnet interaction in Section VII-B is read-only state fetching. We never submit transactions to a live network or move real funds. Extending the harness to a full mainnet-fork node, with real routing and multi-hop paths, is a natural next step.

b) Scale, single annotator, and scoring strictness.: The development split has 120 prompts and the held-out split 87, with a single temperature-0 run per model on two model families. The gold annotations were authored primarily by one team; we provide a second-annotator protocol, a stratified subset, and tooling to compute inter-annotator agreement, but the agreement number awaits an independent pass and is not yet reported. Edge matching is strict and type-level, which penalizes reasonable-but-reordered graphs and makes structural rates a lower bound. This does not affect the unsafe-execution findings, which are driven by missing safety parameterization rather than edge ordering.

c) Scope of the Koan-Safe method.: The enforcement layer improves safety, not intent extraction: it never fabricates a missing amount, price, or chain, so its achievable executable rate is bounded by what the parser and generator read from the prompt. The rule-based generator's fixed presets do not cover structures outside the four categories, which the held-out variants expose; the LLM and hybrid generators mitigate but do not fully remove this. The held-out split was authored by the same team as the method, so it tests structural and surface generalization but not adversarial distribution shift from an independent party.

d) Category scope of the execution layer.: On-chain execution covers swaps and the swap leg of compositional workflows. Limit orders are checked for immediate fillability against the pool mid-price, and cross-chain workflows are not executed because a single local chain cannot bridge. Every unexecuted case is recorded explicitly rather than silently passed.

X. Ethics and Responsible Disclosure

This work is an offline evaluation: no transaction in this paper touches a live network or real funds. The benchmark is intended to help identify unsafe generated workflows before execution, not to encourage executing generated DeFi workflows with real assets. The evaluated no-code system and the language models are used as-is through their public interfaces; we report their failures to motivate safer synthesis, not to single out any implementation. Model outputs and derived metrics are released so the results can be independently reproduced and audited.

Acknowledgements

AI language model assistance was used for grammatical and stylistic edits to the text. All experiments, code, datasets, gold annotations, and quantitative results were designed, implemented, and verified by the authors.

References

[1] S. Qiao *et al.*, "Benchmarking agentic workflow generation," in *International Conference on Learning Representations (ICLR)*, 2025, arXiv:2410.07869.

[2] Y. Zhong, B. Xu, Y. Wang, Z. Shan, S. Qiao, G. Zheng, and N. Zhang, "Chat2Workflow: A benchmark for generating executable visual workflows with natural language," 2026.

[3] Z. Zeng, W. Watson, N. Cho, S. Rahimi, S. Reynolds, T. Balch, and M. Veloso, "FlowMind: Automatic workflow generation with LLMs," in *Proceedings of the Fourth ACM International Conference on AI in Finance (ICAIF)*, 2023, pp. 73–81.

[4] R. Xiao, W. Ma, K. Wang, Y. Wu, J. Zhao, H. Wang, F. Huang, and Y. Li, "FlowBench: Revisiting and benchmarking workflow-guided planning for LLM-based agents," in *Findings of the Association for Computational Linguistics: EMNLP*, 2024.

[5] Y. Qin *et al.*, "ToolLLM: Facilitating large language models to master 16000+ real-world apis," in *International Conference on Learning Representations (ICLR)*, 2024, arXiv:2307.16789.

[6] M. Li *et al.*, "API-Bank: A comprehensive benchmark for tool-augmented LLMs," in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023, arXiv:2304.08244.

[7] Z. Pan, Y. Li, Z. Guan, J. Hu, and Z. Chen, "Intent2Tx: Benchmarking LLMs for translating natural language intents into Ethereum transactions," 2026.

[8] D. Yao, S. Jagannath, B. Aroso, V. Krishnan, and D. Zhao, "Auditable LLM arbiter for DeFi security: A hybrid graph-of-thoughts approach to intent–transaction alignment," in *Workshop on LLM Assisted Security and Trust Exploration (LAST-X), NDSS*, 2026.

[9] H. Adams, N. Zinsmeister, and D. Robinson, "Uniswap v2 core," Uniswap, Tech. Rep., 2020, whitepaper.

[10] S. M. Werner, D. Perez, L. Gudgeon, A. Klages-Mundt, D. Harz, and W. J. Knottenbelt, "SoK: Decentralized finance (DeFi)," in *Proceedings of the 4th ACM Conference on Advances in Financial Technologies (AFT)*, 2022, pp. 30–46.

[11] P. Daian, S. Goldfeder, T. Kell, Y. Li, X. Zhao, I. Bentov, L. Breidenbach, and A. Juels, "Flash boys 2.0: Frontrunning in decentralized exchanges, miner extractable value, and consensus instability," in *IEEE Symposium on Security and Privacy (SP)*, 2020, pp. 910–927.